CIS613 - Assignment: Testing with LLM

Ahsanul Kabir

April 17, 2026

1 Testing using LLM

Created file: *test_llm.py*

Model used: Gemini 3 (Fast)

Chat log: Please click here

Prompt Used:

```
INVALID = -1
SCALENE = 0
ISOSCELES = 1
EQUILATERAL = 2


def triangle_identifier(a, b, c):

    is_a_triangle = None

    # Is A Triangle?
    if (a < b + c) and (b < a + c) and (c < a + b):
        is_a_triangle = True
    else:
        is_a_triangle = False

    # Determine Triangle Type
    triangle_type = INVALID
    if is_a_triangle:
        if (a == b) and (b == c):
            triangle_type = EQUILATERAL
        elif (a != b) and (a != c) and (b != c):
            triangle_type = SCALENE
        else:
            triangle_type = ISOSCELES
    else:
        triangle_type = INVALID

    return triangle_type

Write unit tests for the python script above. The script returns the type of triangle given

the sides of the triangle.
```

2 Types of Tests Identified by LLM

The test types identified by the agentic model are mostly Equivalence Class Testing.

Each method's testing type is analyzed below: The first three tests represent Valid Triangle classes.

- $test_equilateral$: Equivalence Class Testing - Tested the output class of Equilateral triangles.
- $test_isosceles$: Equivalence Class Testing and Decision Table Based Testing - Tested the output class of Isosceles triangles. Besides, it is tested for every input individually, which can be considered as Decision Table Based Testing.
- $test_scalene$: Equivalence Class Testing - Tested the output class of Scalene triangles.

The rest of the tests represent Invalid Triangle classes and the possible combinations within that.

- $test_invalid_triangle_inequality$: Equivalence Class Testing - Tested positive side inequality.
- $test_invalid_zeros$: Boundary Value Testing and Equivalence Class Testing - Tested the lower bound zero.
- $test_invalid_negative$: Equivalence Class Testing - Tested

3 Scores

I chose previously written Equivalence Class Testing (test_equivalence_class.py) and Decision Table Based Testing (test_decision_table.py) suites to compare with the LLM-generated test suite.

3.1 Code Coverage

All three test suites had 100% test coverage.

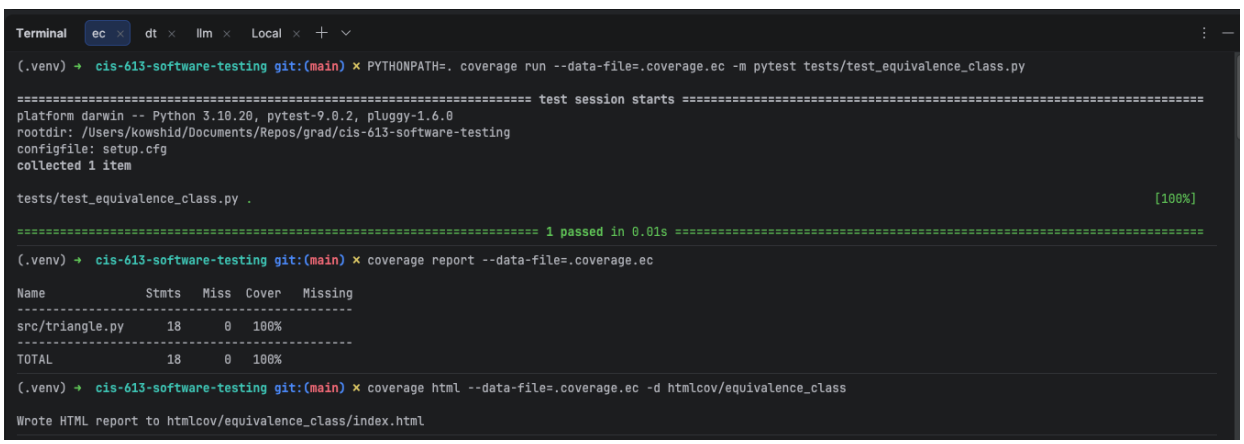
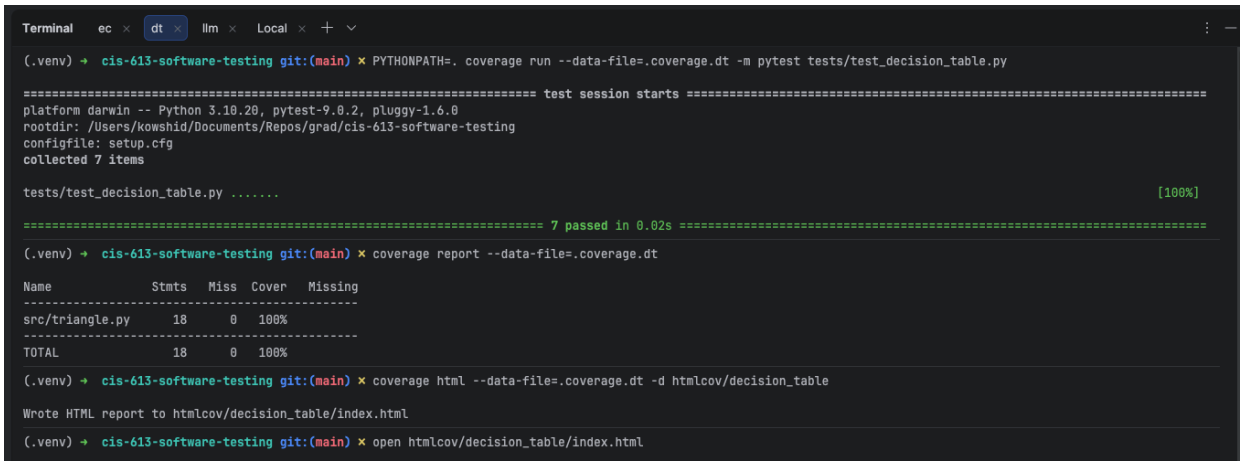
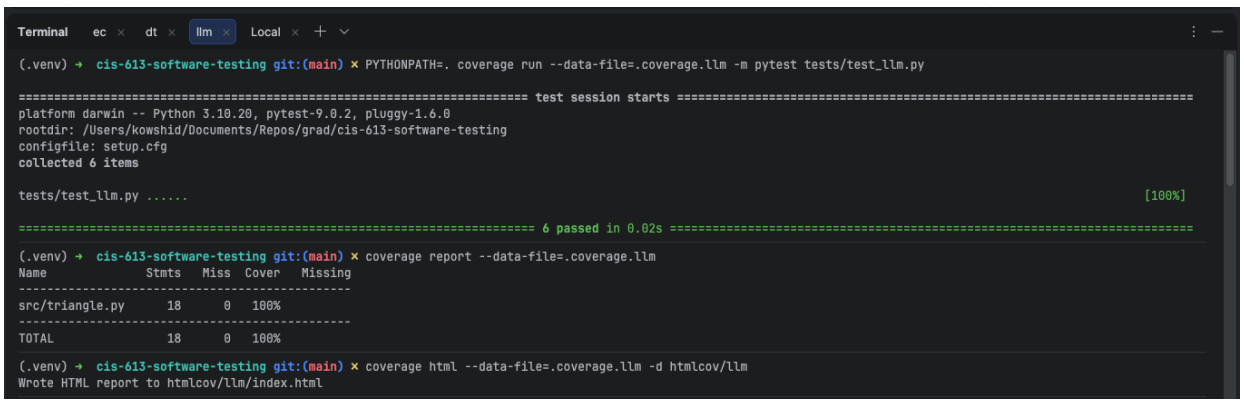


Figure 1: Equivalence Class Testing Coverage



```
Terminal   ec   dt   llm   Local

(.venv) ➜  cis-613-software-testing git:(main) ✗ PYTHONPATH=. coverage run --data-file=.coverage.dt -m pytest tests/test_decision_table.py

============================================================================ test session starts ============================================================================
platform darwin -- Python 3.10.20, pytest-9.0.2, pluggy-1.6.0
rootdir: /Users/kowshid/Documents/Repos/grad/cis-613-software-testing
configfile: setup.cfg
collected 7 items

tests/test_decision_table.py .......                                                                                                                                      [100%]

============================================================================= 7 passed in 0.02s =============================================================================

(.venv) ➜  cis-613-software-testing git:(main) ✗ coverage report --data-file=.coverage.dt

Name              Stmts   Miss  Cover   Missing
-----------------------------------------------
src/triangle.py      18      0   100%
-----------------------------------------------
TOTAL                18      0   100%

(.venv) ➜  cis-613-software-testing git:(main) ✗ coverage html --data-file=.coverage.dt -d htmlcov/decision_table

Wrote HTML report to htmlcov/decision_table/index.html

(.venv) ➜  cis-613-software-testing git:(main) ✗ open htmlcov/decision_table/index.html
```

Figure 2: Decision Table Based Testing Coverage



```
Terminal   ec   dt   llm   Local

(.venv) ➜  cis-613-software-testing git:(main) ✗ PYTHONPATH=. coverage run --data-file=.coverage.llm -m pytest tests/test_llm.py

============================================================================ test session starts ============================================================================
platform darwin -- Python 3.10.20, pytest-9.0.2, pluggy-1.6.0
rootdir: /Users/kowshid/Documents/Repos/grad/cis-613-software-testing
configfile: setup.cfg
collected 6 items

tests/test_llm.py ......                                                                                                                                                  [100%]

============================================================================= 6 passed in 0.02s =============================================================================

(.venv) ➜  cis-613-software-testing git:(main) ✗ coverage report --data-file=.coverage.llm
Name              Stmts   Miss  Cover   Missing
-----------------------------------------------
src/triangle.py      18      0   100%
-----------------------------------------------
TOTAL                18      0   100%

(.venv) ➜  cis-613-software-testing git:(main) ✗ coverage html --data-file=.coverage.llm -d htmlcov/llm
Wrote HTML report to htmlcov/llm/index.html
```

Figure 3: LLM Test Suite Coverage

3.2 Mutation Scores

Formula: $MutationScore = KilledMutants/TotalMutants$

- Equivalence Class Testing: $18/33 = 0.55$
- Decision Table-Based Testing: $18/33 = 0.55$
- LLM Generated Tests: $28/33 = 0.85$

One reason the mutation score is high for the LLM-generated tests is that they use constant values instead of $INVALID$, $SCALENE$, $ISOSCELES$, and $EQUILATERAL$ when comparing the expected and actual return values.

Mutation logs are attached to the zip.

4 Correctness

The expected values defined by the LLM test cases are correct in my case. One thing to consider here is that it created a low number of tests where the scope to hallucinate is low. Besides, I gave just one prompt in the chat, which did not need any context for the agent.

The LLMs mostly create tests that are code-based. For this type of testing, it is also important to use spec-based testing to ensure the solution is correct. In our Triangle problem, for example, we can verify whether the LLM-generated tests

align with the formal definitions of equilateral, isosceles, and scalene triangles. I used an online verifier to check the side values, and in my case, all of them are valid.

5 Conclusion

I did not find any new type of test or new test case that's not already covered by the previous test suite I had written. One thing to consider here can be specifying the test type for the agent, like "*Robust Worst-Case Boundary Value Testing*", and checking whether the agent can generate the tests or how it behaves when the resulting test suite becomes very large.